

EduGenie

Google Gemini Powered Learning Assistant

Phase-Wise Detailed Project Documentation

PROJECT OVERVIEW

EduGenie is a Google Gemini powered learning assistant designed to support students through Question Answering, simple Concept Explanation, Quiz Generation, Summarization, and Learning Recommendations. The project is organized into five sequential development phases.

Overall Project Flow

Phase 1: Model Selection & Architecture → Phase 2: Core Module Development → Phase 3: Frontend + Backend Integration → Phase 4: Testing & Execution → Phase 5: Documentation & Final Output.

PHASE 1 – MODEL SELECTION AND SYSTEM ARCHITECTURE

1.1 Objective

Select the AI technology, define the system architecture, configure the AI service, and prepare the project structure before development.

1.2 AI Model Selection

- Google Gemini for Question Answering.
- Google Gemini for Quiz Generation.
- Google Gemini for Summarization.
- Google Gemini for Learning Recommendations.
- A simplified approach for student-friendly concept explanations.
- The detailed project document identifies LaMini-Flan-T5-783M as the local model for concept explanation.

1.3 Gemini API Configuration

- Configure access to the Gemini API.
- Generate and securely store the API key.
- Connect the backend to the AI service.
- Send user requests and receive generated responses.

1.4 Folder Architecture

EduGenie/

■■■ main.py

■■■ config.py

■■■ ai_client.py

■■■ explanation_module.py

■■■ qna.py

■■■ quiz_module.py

■■■ summary_module.py

■■■ learning_path.py

■■■ templates/index.html

■■■ static/style.css, app.js

1.5 Output

System architecture, AI model selection, API configuration, and project folder structure.

PHASE 2 – CORE FUNCTIONALITY DEVELOPMENT

2.1 Objective

Implement the main educational features as separate Python modules for easier organization, testing, and maintenance.

2.2 Explanation Module

Accepts a topic and generates a simple, student-friendly explanation.

2.3 Q&A; Module

Accepts an academic question and generates a concise AI-based answer.

2.4 Quiz Module

Accepts a topic and generates multiple-choice questions with correct answers.

2.5 Summary Module

Converts long educational text into a shorter, easy-to-understand summary.

2.6 Learning Path Module

Provides structured learning recommendations from beginner to advanced level.

2.7 Output

Working Python modules for explanation, Q&A;, quiz, summary, and learning recommendations.

PHASE 3 – BACKEND AND FRONTEND INTEGRATION

3.1 Objective

Connect the AI modules to a browser-based interface through FastAPI.

3.2 FastAPI REST Endpoints

Endpoint	Function
/qa	Question Answering
/explain	Concept Explanation
/quiz	Quiz Generation
/summarize	Summarization
/learn/recommendations	Learning Recommendations

3.3 Frontend

- index.html provides task selection, text input, Generate button, and result area.
- style.css controls layout, buttons, fields, results, and responsive design.
- app.js sends requests to FastAPI and displays the AI response.

3.4 System Flow

User → Frontend → FastAPI Endpoint → Python Module → AI Service → Response → Frontend → User

3.5 Output

A browser-based EduGenie interface connected to the FastAPI backend.

PHASE 4 – TESTING AND EXECUTION

4.1 Objective

Verify that EduGenie functions correctly and handles common error conditions.

4.2 Local Execution

- Open the project in VS Code.
- Install required Python packages.
- Create the .env file.
- Add the Gemini API key.
- Start the FastAPI server using Uvicorn.
- Open the local URL in a browser.

4.3 Functional Testing

Function	Test Input	Expected Output
Q&A	Academic question	AI-generated answer
Explanation	Topic	Simple explanation
Summary	Long paragraph	Short summary
Quiz	Topic	MCQ quiz
Learning Path	Subject	Beginner-to-advanced recommendations

4.4 Error Handling

- Missing API key
- Invalid API key
- Temporary AI/API failure
- Invalid or empty user input

4.5 Output

A tested and working EduGenie application.

PHASE 5 – DOCUMENTATION AND FINAL OUTPUT

5.1 Objective

Collect project outputs, screenshots, testing results, and prepare the final project report.

5.2 Required Screenshots

- EduGenie Home Page
- Question Answering
- Concept Explanation
- Quiz Generation
- Summary
- Learning Recommendation

5.3 Final Documentation

- Introduction
- Problem Statement

- Objectives
- Existing System
- Proposed System
- System Requirements
- Architecture
- Modules
- Testing
- Screenshots / Output
- Conclusion
- Future Scope
- References

5.4 Output

Complete project documentation with screenshots, outputs, testing details, conclusion, future scope, and references.

PHASE-WISE SUMMARY

Phase	Main Work	Output
1	Model selection & architecture	System architecture
2	Core AI modules	Working modules
3	Frontend + Backend	Working web application
4	Testing & execution	Tested application
5	Documentation & final output	Final project documentation

CONCLUSION

EduGenie follows a clear five-phase development structure, progressing from AI model selection and system architecture to core module development, web integration, testing, and final documentation.